Seguridad — Guía 7

RCCD 2C 2026 · Resumen de la teórica 14 (Seguridad + Blockchain), la práctica de seguridad (13/10) y la práctica de los ej. 6 y 9. Notación de la guía: $E_Q(M)$ cifra, $D_Q(C)$ descifra, $H(M)$ hash, K simétrica, K^+_A pública de A, K^-_A privada de A.

0 · La idea en criollo

- **Simétrica** = un **candado con una sola llave** y una copia para cada uno. Es rápida, pero ¿cómo le hacés llegar la copia al otro sin que nadie la vea?
- **Asimétrica** = un **buzón**: cualquiera puede meter una carta (clave **pública**), pero solo el dueño la abre (clave **privada**). Y al revés: lo que el dueño "sella" con su privada, cualquiera lo puede comprobar con la pública. Eso es una **firma**.
- **Hash** = la **huella digital** del mensaje: chiquita, fija, y si cambiás una letra cambia toda.
- **Certificado** = el **DNI de una clave pública**: un tercero en quien todos confían (la CA) firma "esta clave es de fulano".
- **TLS** = usar lo asimétrico (lento) solo para **presentarse y acordar** una clave simétrica, y después hablar con lo simétrico (rápido).

> **La pregunta de todos los ejercicios:** ¿**quién** tiene que probar **qué**, y con **qué clave**?
>
> 1. **¿Qué piden?** Subrayá las propiedades: confidencialidad, integridad, autenticación, no repudio.
> 2. **¿Quién demuestra y quién verifica?** Fijate "del lado de quién" pide el enunciado.
> 3. **¿Con qué clave?** El que demuestra usa **su privada**. El que verifica necesita **la pública** del otro y tiene que **confiar** en ella (certificado o carga previa por canal seguro).

1 · Propiedades (marco de trabajo)

Propiedad	Qué significa	Con qué se logra
Confidencialidad	Solo las partes autorizadas entienden el mensaje.	Cifrar: simétrica con K, o E_{K^+B} (pública del **receptor**).
Integridad	Si lo modifican (o lo reemplazan, repiten o demoran), se detecta.	Hash **protegido con una clave**: MAC o firma. Un hash solo no alcanza, porque el atacante lo recalcula.
Autenticación	Sé quién lo mandó. Nadie se hace pasar por otro.	Firma, MAC, challenge-response, certificados.
No repudio	Puedo **probarle a un tercero** quién lo mandó. El emisor no puede negarlo.	**Firma digital** con la privada del emisor (y certificado que la ate a su identidad).
Autorización	Solo ciertos usuarios hacen ciertas acciones.	Control de acceso, después de autenticar.
Disponibilidad	El servicio atiende (contra DoS).	**No** se logra con criptografía.
Originalidad / Temporalidad	No es una copia repetida / no fue demorado.	Nonce / timestamp **dentro de lo firmado**.

- **No repudio** ⇒ **autenticación**, pero no al revés. Un MAC autentica, pero **no** da no repudio, porque los dos tienen la misma clave.
- El no repudio es en el fondo un concepto **legal**: la tecnología aporta la evidencia (la firma guardada).
- Ante cada tecnología, preguntate qué subconjunto de propiedades provee.

2 · Criptografía clásica (ej. 1)

- **Cifrado** = transforma carácter a carácter o bit a bit, sin mirar el idioma. **Código** = reemplaza palabras enteras (por ejemplo, los navajos en la Segunda Guerra).
- **Sustitución**: cada letra se reemplaza por otra y se mantiene el orden.
  - **César**: rotar el alfabeto k lugares, así que hay solo 25 claves.
  - **Monoalfabética**: cualquier permutación, 26! ≈ $4 \cdot 10^{26}$ claves. Aun así se rompe por **análisis de frecuencias**, porque cada letra del texto plano siempre se mapea a la misma letra cifrada.
  - **Polialfabética**: un alfabeto distinto por posición, lo que aplana las frecuencias.
- **Transposición**: se reordenan las letras sin enmascararlas (columnas numeradas por una clave tipo MEGABUCK). Las frecuencias quedan iguales a las del idioma.
- Estrategias para romper un César:
  1. **Fuerza bruta**: probar los 25 desplazamientos y quedarse con el que da texto legible.
  2. **Frecuencias**: la letra más frecuente del criptograma probablemente sea la "e" (en inglés y en castellano). El desplazamiento es la distancia entre ambas.
  3. **Palabras cortas y patrones**: las palabras de 1 a 3 letras repetidas (a, the, and, of / de, la, el), los dobles (ll, ss) y la firma al final.
- **Principio de Kerckhoffs**: el algoritmo es **público** y solo la **clave** es secreta. La "seguridad por oscuridad" no funciona.
- **Factor de trabajo**: crece **exponencialmente** con el largo de la clave. Hoy 128/256 bits es uso comercial y ≥ 256 bits es para defensa.
- **One-time pad**: clave aleatoria tan larga como el mensaje, combinada con XOR y usada una sola vez. Es **inviolable** (Shannon), pero **impráctica** por la distribución de la clave.

3 · Simétrica

- Una **única clave K** compartida: $C = E_K(M)$, $M = D_K(C)$. Da **confidencialidad**.
- **DES** (IBM 1977): bloques de 64 bits, 16 rondas con subclaves derivadas de una misma clave. Hoy está obsoleto (clave corta).
  - **3DES**: $C = E_{K3}(D_{K2}(E_{K1}(M)))$.
  - **AES** (128/192/256) es lo mínimo hoy.
- Cifrado de bloque iterativo = rondas de **sustitución + permutación** sobre bloques de n bits.
- Problemas:
  - La **distribución de la clave** es el punto débil histórico.
  - Sola, **no da integridad**: el atacante puede alterar bits del criptograma.
  - Tampoco da **autenticación individual**, porque los dos tienen K.
- **KDC** (Key Distribution Center): cada uno comparte una clave con el KDC.
  - A elige una clave de sesión K_S y manda $A, K_A(B, K_S)$.
  - El KDC le reenvía a B $K_B(A, K_S)$.

4 · Asimétrica (clave pública)

- Cada uno tiene un par: K^+ (pública, se reparte) y K^- (privada, nunca sale de su máquina). No se puede derivar la privada a partir de la pública.
- Historia: Diffie-Hellman (1976) y RSA (1978). El acuerdo de clave Diffie-Hellman permite que dos partes acuerden un secreto por un canal público.
- **Propiedad clave de RSA**: $K^-(K^+(m)) = m = K^+(K^-(m))$. Sirve en los dos sentidos.
- RSA, generación de claves:

1. Elegir primos grandes p, q.
2. Calcular $n = p \cdot q$ y $z = (p-1)(q-1)$.
3. Elegir e coprimo con z.
4. Calcular d tal que $e \cdot d \bmod z = 1$.
5. La pública es (n, e) y la privada (n, d).

- RSA, uso: $c = m^e \bmod n$ y $m = c^d \bmod n$. Ejemplo de juguete: p=3, q=11 → n=33, e=3, d=7. La seguridad viene de que factorizar n es caro.
- Desventajas:
  - Es **mucho más lenta** que la simétrica, así que se usa sobre datos chicos (hashes, claves).
  - El problema pasa a ser si la K^+ que tengo es realmente de quien dice. Eso lleva a §6.

> **Para que solo B lo lea**
> uso algo de B que todos tienen, su **pública**:
> $C = E_{K^+_B}(M)$ → **confidencialidad**

> **Para probar que fui yo (A)**
> uso algo mío que solo yo tengo, mi **privada**:
> $F = E_{K^-_A}(H(M))$ → **autenticación + integridad + no repudio**

> **Error típico:** "cifro con la privada para que sea confidencial". **No**: lo que cifrás con tu privada lo descifra cualquiera con tu pública. Eso es firmar, no ocultar.

Híbrido (lo que se usa en la práctica): la asimétrica sirve para autenticar y para mandar o acordar una clave de sesión simétrica, y los datos van cifrados con esa simétrica (AES). Así funciona TLS, y también PGP para mail.

5 · Hash, MAC y firma digital

Función de hash criptográfica

- Toma una entrada de cualquier tamaño y da una salida de **tamaño fijo**. Es fácil de calcular y tiene **efecto avalancha** (una letra distinta cambia todo el resultado).
- Tres propiedades:
  - **Preimagen** (no inversible): dado y, es inviable hallar x con H(x)=y.
  - **Segunda preimagen**: dado x, es inviable hallar otro x′ con el mismo hash.
  - **Colisiones**: es inviable hallar cualquier par x≠x′ con el mismo hash.
- Algoritmos: **MD5** (128 bits) y **SHA-1** (160 bits) tienen colisiones encontradas, así que están rotos. Hoy se usa **SHA-2** (SHA-256/512) o SHA-3.
- Un hash **solo no protege**: el atacante cambia M y recalcula H(M). Hay que atarlo a una clave.

MAC (hash con clave compartida)

- $mac = HMAC_K(M)$. El receptor recalcula el MAC y compara.
- Da **integridad + autenticación**, porque sin K no se puede fabricar un MAC válido.
- **No da no repudio**: el receptor también tiene K, así que el emisor puede decir "eso lo fabricaste vos".

Firma digital

```
A firma:     F = E_{K-A}( H(M) )            → manda  M + F
B verifica:  D_{K+A}(F)  ==  H(M) ?          sí → íntegro y de A
```

- Se firma el **hash** y no el mensaje porque la asimétrica es cara sobre datos largos, y el hash es corto y de tamaño fijo.
- Da **integridad, autenticación y no repudio**. **No da confidencialidad**: M viaja en claro.
- Ejemplo: RSA + SHA-256 (antes RSA + SHA-1). En blockchain se usa ECDSA/Schnorr.

Firma + confidencialidad (esquema del ej. 2)

```
Emisor A:
  1. D = H(M)                                      (hash, no lleva clave)
  2. F = E_{K-A}(D)                                (firma con la PRIVADA de A)
  3. C = E_{K+B}(M + F)                            (cifra con la PÚBLICA de B)
     (en la práctica, híbrido: C = E_K(M+F) junto con E_{K+B}(K))
Receptor B:
  1. M + F = D_{K-B}(C)                            → solo B puede → confidencialidad
  2. D_{K+A}(F) == H(M) ?                          → no repudio + integridad
```

6 · El problema de la clave pública: MITM y certificados (ej. 3)

Ataque man-in-the-middle en el intercambio de claves

1. B publica su K^+_B. X la intercepta y le entrega a A **su propia** K^+_X. Hace lo mismo en la otra dirección.
2. A cifra $E_{K^+X}(M)$ creyendo que es para B.
3. X descifra, lee o **modifica** M, lo vuelve a cifrar con K^+_B y se lo pasa a B. Nadie nota nada.

Solución: un **tercero en quien todos confían** firma el vínculo **identidad ↔ clave pública**. Eso es un certificado.

Certificado digital (X.509v3)

```
Cert_U = ( U ; K+_U ; F )     con   F = E_{K-CA}( H(U + K+_U) )
```

- Contiene:
  - **Sujeto** (quién es U) y **su clave pública**.
  - **Emisor** (qué CA lo firmó).
  - **Validez** (desde/hasta) y **número de serie**.
  - Algoritmos, y la **firma de la CA**.
- Para verificarlo:
  1. Comprobar que $D_{K^+CA}(F) == H(U + K^+_U)$.
  2. Comprobar que está vigente y que no fue revocado (**CRL / OCSP**).
  3. Tener la K^+_{CA}, que ya **estaba instalada** (es la **raíz de confianza**: navegador o sistema operativo, o la CA propia de la organización instalada a mano).
- **PKI** = el esquema jerárquico y centralizado: CA raíz → CAs intermedias → certificado. Se verifica **la cadena** hasta una raíz confiable.
- **Web of Trust** (PGP) = esquema descentralizado: los usuarios se firman las claves entre sí ("fiestas de firmado").

> **La idea más importante:** mandar el certificado **no prueba nada**. Es público y cualquiera lo copia. Lo que prueba la identidad es **firmar algo nuevo con la privada** (un challenge, o el resumen del handshake).

7 · Autenticación: challenge-response y replay (ej. 5)

Con clave compartida K_{AB} (autenticación de dos vías)

```
1. A → B : A                    (identidad)
2. B → A : R_B                  (challenge aleatorio grande, en claro)
3. A → B : K_AB(R_B)            → B sabe que es A
4. A → B : R_A
5. B → A : K_AB(R_A)            → A sabe que es B   (después A puede mandar K_AB(K_S))
```

- **Versión simplificada en 3 mensajes** (`A, R_A / R_B, K_AB(R_A) / K_AB(R_B)`): es vulnerable al **ataque por reflexión** si se permiten sesiones paralelas.
  1. T, haciéndose pasar por A, abre la sesión 1.
  2. Cuando B le manda su challenge R_B, T abre una sesión 2 y le manda **ese mismo R_B** como challenge propio.
  3. B responde $K_{AB}(R_B)$ en la sesión 2. T usa esa respuesta en la sesión 1 y queda autenticado como A.
- Reglas de diseño:
  - El que inicia se identifica primero.
  - Usar claves distintas por sentido (K_{AB} y K'_{AB}).
  - Sacar los challenges de conjuntos distintos (por ejemplo, uno pares y el otro impares).
  - Que el protocolo resista sesiones paralelas.

Con clave pública (como SSH)

- El servidor manda un challenge R. El cliente responde $E_{K^-C}(R)$, firmando con su privada. El servidor lo verifica con la K^+_C que tiene **cargada de antemano**.

Replay (ataque de repetición)

El atacante no rompe nada: **graba un mensaje válido y lo reenvía**. La firma sigue siendo correcta. Defensas:

- **Nonce**: valor aleatorio de un solo uso. O lo elige el que verifica (challenge), o el verificador recuerda los nonces que ya vio.
- **Timestamp**: se acepta el mensaje solo dentro de una ventana corta (±5 min), con relojes sincronizados.
- **El nonce y el timestamp van dentro de lo firmado**. Si no, el atacante los cambia.

8 · ¿Dónde se instala cada clave? (tabla para los ejercicios de diseño)

Escenario	El que se autentica tiene	El que verifica tiene
Autenticar N clientes con claves sueltas (SSH)	Cada cliente C_i: su par (K^+_{Ci}, K^-_{Ci}), o sea **N pares**.	Servidor: las **N públicas** K^+_{Ci}, asociadas a cada usuario (authorized_keys).
Autenticar el servidor con claves sueltas	Servidor: su par (K^+_S, K^-_S).	Cada cliente: K^+_S (known_hosts).
Con certificados de una CA	Su par + **su certificado firmado por la CA**.	Solo el **certificado de la CA** (K^+_{CA}): **una sola clave** en lugar de N, y las altas y bajas no tocan al verificador.

- **Contraseña sobre HTTPS**: se puede robar, adivinar o reutilizar.
- **Claves públicas sueltas**: el verificador tiene que guardar N claves y hacer las altas y bajas a mano.
- **Certificados + TLS mutuo**: es lo que escala.

9 · SSL/TLS (ej. 4)

- Va **sobre TCP**, por debajo de la aplicación (HTTPS = HTTP sobre TLS, puerto 443). Es **orientado a conexión**.
- Tiene dos subprotocolos: el **handshake** (establecer la conexión segura) y el **record** (usarla).
- SSL fue el primero y TLS es su sucesor. No interoperan directamente, por eso se habla de "SSL/TLS".

Pasos

1. **HELLO**: se negocian versión, conjunto de algoritmos (cipher suite) y parámetros. Cada lado manda un **valor al azar** (nonce).

2. **Autenticación**: el servidor manda su **certificado**. Opcionalmente pide el del cliente (TLS mutuo).
3. **Key exchange**: el cliente genera la **pre-master secret** y la manda cifrada con la **pública del servidor** (sacada del certificado). Solo el servidor la puede leer.
4. Cada lado calcula por su cuenta la **master key** = f(pre-master, $nonce_C$, $nonce_S$), y de ahí las claves **simétricas de sesión**.
5. **Data sharing**: todo viaja como **criptograma** con clave simétrica (rápida), más un MAC por registro.

Parámetro	Etapa
Versión de protocolo · Conjunto de algoritmos · Parámetros criptográficos	HELLO
Valor al azar (nonce)	HELLO (se manda) y Key-Exchange (se usa para derivar la master)
Certificado del peer · Clave maestra	Key-Exchange
Criptograma	Data-sharing

Diagrama de secuencia (con TLS mutuo)

```
Cliente                                              Servidor
  |-- ClientHello: versión, suites, R_C ---------------->|  en claro
  |<-- ServerHello: suite elegida, R_S -----------------|  en claro
  |<-- Certificate: Cert_S ----------------------------|  en claro
  |<-- CertificateRequest  (solo si TLS mutuo) ---------|  en claro
  |   (cliente valida Cert_S con K+_CA)                 |
  |-- Certificate: Cert_C  (si se lo pidieron) -------->|  en claro
  |-- ClientKeyExchange: E_{K+S}(PMS) ----------------->|  cifrado asimétrico
  |-- CertificateVerify: E_{K-C}(H(handshake)) -------->|  firma = prueba de posesión
  |   ambos: master = f(PMS, R_C, R_S) → claves de sesión
  |-- ChangeCipherSpec + Finished --------------------->|  cifrado simétrico
  |<-- ChangeCipherSpec + Finished --------------------|  cifrado simétrico
  |== datos de aplicación (HTTP) cifrados con K_sesión ==|
```

- **Finished** lleva un MAC de todo el handshake, así se detecta si alguien tocó los mensajes en claro (por ejemplo, para bajar la suite).
- Se pasa a una clave **simétrica** porque es mucho más rápida, y porque es una clave nueva por sesión: si alguien graba la sesión, no le sirve para otras.
- Los **nonces** R_C y R_S evitan el replay de un handshake viejo.

TLS mutuo (ej. 6 y 7)

- Normalmente **solo el servidor** presenta certificado, y el cliente se autentica después con usuario y contraseña dentro del túnel.
- Si el servidor quiere autenticar al cliente con certificados, el handshake suma **CertificateRequest** → **Certificate** → **CertificateVerify**.
- **CertificateVerify** (la firma del cliente sobre el handshake, nonces incluidos) es lo que prueba que tiene la privada.
- El verificador chequea cuatro cosas:
  1. ¿Lo firmó nuestra CA?
  2. ¿Está vigente y no revocado?
  3. ¿Tiene la privada? (verifica el CertificateVerify)
  4. ¿Está autorizado? (por ejemplo, si pertenece al grupo habilitado)

  Si falla algo, corta el handshake.

> **Cómo leer cada caso:**
>
> - **Hay que autenticar al servidor (o al proxy)**: el servidor tiene su certificado y su privada, y los clientes tienen el certificado de la CA. Con eso los clientes evitan un proxy falso y, como la PMS va cifrada con la pública verdadera, además hay **confidencialidad**. Ese es el caso del hotel.
> - **Hay que autenticar a los usuarios**: se suma TLS mutuo. Cada usuario tiene su certificado y su privada, y el proxy tiene el certificado de la CA.

10 · Seguridad por capas

Capa	Protocolo	Qué da
Aplicación	SSH, PGP/S-MIME (mail), firmas en la URL de una API	Extremo a extremo de verdad: "la máxima seguridad se logra de extremo a extremo".
Transporte	SSL/TLS (HTTPS, SMTPS, IMAPS)	Canal cifrado y autenticado por conexión.
Red	IPsec: **AH** (integridad + autenticación + no repudio) · **ESP** (confidencialidad)	Transparente para las aplicaciones (VPNs).
Enlace	WEP (roto) · WPA · WPA2	Protege solo el salto inalámbrico.

Ninguna red es 100% segura. La gestión de seguridad de la información se rige por la familia de normas **ISO 27000**.

11 · Diseñar sin TLS ni sesiones: firmar cada pedido (ej. 8 y 9)

El método de la clase es partir de la versión más ingenua e ir preguntando, en cada paso, qué hace el atacante:

Versión	Qué se agrega	Aut.	Integ.	No rep.	Replay	Por qué falla
V0	nada	×	×	×	×	El servidor no sabe quién pidió.
V1	`clientId=33`	×	×	×	×	Decir quién sos no es probarlo: cualquiera escribe 33.
V2	`pass=secreto`	×	×	×	×	Viaja en claro: la leen y la reutilizan, y pueden cambiar la fruta.
V3	`mac=HMAC_K(...)`	✓	✓	×	×	El servidor también tiene K: "eso lo fabricó él".
V4	`sig=E_{K-C}(H(...))`	✓	✓	✓	×	Copian la URL y la reenvían mil veces.
V5	`ts` + `nonce` **dentro** de la firma	✓	✓	✓	✓	—

- **Cliente**:
  - Tiene su par de claves (la privada no sale de su máquina), su id y los algoritmos acordados (RSA + SHA-256).
  - Arma una **cadena canónica** con todos los parámetros en un orden fijo, calcula `FIRMA =` $E_{K^-C}(H(\text{cadena}))$ y la agrega a la URL en Base64URL.
- **Servidor**:
  - Tiene una tabla id → K^+_C cargada antes por canal seguro (o certificados), el reloj sincronizado y los nonces recientes. **No necesita privada propia**.
  - Busca la K^+, chequea ts y nonce, recalcula el hash, verifica la firma y **guarda pedido + firma** como evidencia para el no repudio.

- **Mensajes internos (ej. 8)**: es lo mismo aplicado a mensajes.
  - Cada empleado tiene su par de claves y un **certificado de la CA de la empresa** que lo ata a su identidad.
  - Firma cada mensaje. Para que no pueda negarlo, el mensaje incluye fecha y destinatario.
  - Se guarda el mensaje firmado.
  - Un MAC con clave compartida no alcanza, porque el que lo recibe también lo podría haber generado.

12 · Blockchain (teórica, sin guía)

- **Qué agrega a la firma**: la firma da autenticidad, integridad y no repudio, pero **no da orden**.
  - No dice cuál de dos mensajes fue primero.
  - No garantiza unicidad.
  - No garantiza que todos vean el mismo historial.
- **Doble gasto**: Alice firma "moneda #42 → Bob" y también "moneda #42 → Carol". Las dos firmas verifican, y la criptografía no puede decidir cuál vale. Clásicamente lo resuelve un tercero (el banco).
- **Blockchain** = registro **append-only**, replicado entre nodos que **no confían entre sí**, que acuerdan **un único orden sin autoridad central**. Combina **hash** → **integridad**, **firmas** → **autoría** y **consenso** → **orden**.
- **Cadena de hashes**:
  - Cada header lleva `prev = H(bloque anterior)`, la raíz de Merkle, el timestamp y el nonce.
  - Si alterás una transacción, cambia el hash de ese bloque, deja de coincidir el `prev` del siguiente y se invalida toda la cadena posterior.
- **Árbol de Merkle**:
  - La raíz (32 bytes) va en el header.
  - Para probar que una tx está incluida alcanzan **$\log_2(n)$** hashes. Así verifican los clientes livianos (**SPV**) sin bajar el bloque entero.
- **Identidad**:
  - Dirección = hash(clave pública). No hay registro de usuarios.
  - Una tx gasta salidas previas (**UTXO**) y se firma con la privada (ECDSA/Schnorr, curva secp256k1).
  - Cada nodo verifica la firma y que la salida exista y no esté gastada.
- **Consenso**:
  - **Generales bizantinos** (Lamport, Shostak, Pease, 1982): con f nodos maliciosos hacen falta **$n \geq 3f+1$** participantes conocidos.
  - En una red abierta esto falla por el **ataque Sybil**: crear identidades es gratis, así que "un nodo, un voto" no sirve.
  - La solución de Nakamoto (2008) es **votar con un recurso escaso**: cómputo.
- **Proof of Work** (Bitcoin):
  - Hay que encontrar un nonce tal que `SHA256(SHA256(header)) < target`. Es caro de producir y barato de verificar (un solo hash).
  - El target se reajusta cada 2016 bloques para sostener ~10 min por bloque.
  - Gana la cadena **con más trabajo** acumulado. Minar es una lotería con probabilidad proporcional al cómputo.
- **Confirmaciones**:
  - Si el atacante tiene $q < 50\%$ del cómputo, su probabilidad de alcanzar a la cadena honesta cae exponencialmente con z bloques. Por ejemplo, con $q=10\%$ alcanzan $z=5$ para $P<0{,}1\%$.
  - Con **$q \geq 50\%$** (ataque del 51%) gana siempre.
  - "6 confirmaciones" (~1 h) es una convención, no un teorema.
- **La mirada de redes**:
  - La difusión es por **gossip** (cada nodo reenvía a sus pares).

- Hay **forks transitorios** cuando dos mineros encuentran un bloque casi a la vez. Se resuelven por la cadena con más trabajo y el otro bloque queda **huérfano**.
- Bloques más rápidos ⇒ más forks y más trabajo perdido (latencia vs intervalo). Bitcoin hace ~7 tx/s.

	Proof of Work	**Proof of Stake**	**BFT (PBFT)**
Recurso escaso	Cómputo (energía)	Capital en garantía (slashing si hace trampa)	Identidad conocida
Red	Abierta	Abierta	Permisionada
Finalidad	Probabilística	Económica, por checkpoints	Inmediata
Tolera	< 50% del cómputo	< 1/3 del stake	f < n/3 nodos
Ejemplo	Bitcoin	Ethereum (desde 2022)	CometBFT (Cosmos)

- **Ethereum** funciona como **máquina de estados replicada**: $\texttt{estado}_{n+1} = \delta(\texttt{estado}_n, \texttt{txs})$ (Schneider 1990), pero entre nodos que no confían entre sí.
  - **Smart contracts**: el código corre en la EVM.
  - **Gas**: cada instrucción cuesta, lo que acota el cómputo y corta los loops infinitos.
  - Los bugs son **inmutables** (caso The DAO, 2016).
- **Cuándo conviene una blockchain**: cuando muchos escriben, no confían entre sí, no hay un tercero aceptable para todos y el historial debe ser verificable públicamente. Si hay un tercero confiable, **una base de datos replicada es más simple, rápida y barata**.

13 · Mapa de la guía

Ej.	**Tema**	**Mirá**
1	Romper un César: estrategia	§2 (fuerza bruta / frecuencias / palabras cortas)
2	Firma + confidencialidad con el esquema de cajas	§4 regla de claves, §5 esquema
3	MITM en el intercambio de públicas, qué tiene un certificado	§6
4	Parámetros → etapa de TLS, diagrama de secuencia	§9
5	SSH challenge-response: dónde van las claves; versión con certificados	§7, §8
6	Proxy HTTPS que autentica usuarios	§8, §9 TLS mutuo
7	WiFi de hotel: autenticidad del proxy + confidencialidad	§9 (el 6 al revés)
8	No repudio en mensajes internos	§5, §11
9	API HTTP sin cifrar ni sesiones: integridad + no repudio	§11 (V0→V5)